

KRAKEN

Analisis General del Sistema

Agente IA Autonomo + Escaner de Vulnerabilidades + Defensa en Profundidad

100% Rust | 20+ Crates | 111,000+ Lineas | 1,594 Tests | 9/10 Fases Hardening
Claw Code Venezuela | Julio 2026

CONFIDENCIAL | Documento generado automaticamente desde el codigo fuente

1. Funciones del Sistema

1.1. Arquitectura General

Kraken es un agente de IA autonomo para codigo con seguridad incorporada, implementado como un binario estatico de Rust (~225 MB en debug) sin dependencias externas de runtime. El workspace contiene 18 crates (2 binarios, 16 librerias) que totalizan 111,053 lineas de Rust en 176 archivos fuente, mas 5 targets de fuzzing.

1.2. Funciones Principales

- REPL interactivo: Terminal con historial, autocompletado, 140+ comandos slash
- Prompts one-shot: kraken prompt 'haz X' para automatizacion directa
- 44+ herramientas de agente: read_file, write_file, edit_file, bash, grep, glob, WebSearch, WebFetch, Agent, TodoWrite, NotebookEdit, Skill, ToolSearch, PlanMigration, BatchEdit, VerifyMigration y mas
- Multi-proveedor: Anthropic, DeepSeek, xAI, OpenAI, DashScope, Ollama, LM Studio, Big Pickle
- Proveedores gratuitos: DeepSeek (5M tokens/mes), Big Pickle (ilimitado), Ollama (local)
- Memoria persistente: Notas en .kraken/memory/ con WriteNote/ReadNote/ListNotes
- Orquestacion multi-agente: Sub-agentes paralelos, equipos, delegacion con contexto
- Sesiones: Checkpoint, resume, compactacion automatica, exportacion JSON
- MCP (Model Context Protocol): Servidores, herramientas, recursos
- Plugins: Sistema instalable con ciclo de vida completo
- Modo offline: Cola SQLite con sincronizacion automatica
- Vision/Imagenes: Captura de pantalla y lectura de imagenes

1.3. Escaner de Vulnerabilidades (vulnscan)

El crate vulnscan (~7,686 lineas, 41 archivos) realiza analisis multi-capa combinando deteccion estatica por patrones con analisis potenciado por LLM.

- Analisis estatico en 9 lenguajes: C, C++, Rust, Go, Java, JavaScript/TypeScript, Python, Ruby, Swift
- 18 clases de vulnerabilidad: SQLi, XSS, Command Injection, Buffer Overflow, Use-After-Free, Double-Free, Integer Overflow, Crypto Flaws, Auth Bypass, IDOR, CSRF, SSRF, Path Traversal, XXE, Open Redirect, Hardcoded Secrets, Supply Chain, Race Conditions
- Pipeline de caceria: Recon -> Discover -> Chain -> Exploit -> Report
- 3 modos: fast (regex), deep (patrones + LLM), overnight (pipeline autonomo completo)
- 7 system prompts LLM especializados por clase de vulnerabilidad
- Encadenamiento BFS: grafo de ataque con rutas mas cortas hacia RCE
- Generacion de exploits por LLM y por template

1.4. OSINT Framework

El crate osint (~4,539 lineas, 11 modulos) proporciona inteligencia de fuentes abiertas.

- DNS: Resolucion A, AAAA, MX, TXT, NS, SOA, CNAME, WHOIS
- Email: Validacion MX, deteccion de proveedor, HIBP API v3
- Social: 70+ plataformas, SocialSearcher, ProfileExtractor
- Personas: NameParser, PhoneOSINT (100+ paises), IdentityCorrelator
- Infraestructura: PortScanner, CertTransparency (crt.sh), ASNLookup, TechFingerprinter, IPEnricher (Shodan, ipinfo, rDNS)

- Reportes: JSON, HTML dark-mode, Markdown, CSV, Text

1.5. Optimizacion

- Particle Swarm Optimization (PSO)
- Algoritmo Genetico (GA)
- Optimizacion de Colonia de Hormigas (ACO)
- Recocido Simulado (SA)

2. Arquitectura del Sistema

2.1. Diagrama de Capas

El sistema se organiza en 6 capas verticales, desde la interfaz de usuario hasta el aislamiento del sandbox, con una capa transversal de seguridad que abarca las fases 1-9 de hardening.

```
+-----+
|          CLI / REPL (rusty-clang-cli)          |
|          ~15,584 lineas, 4 archivos fuente      |
+-----+
|  commands (140+ cmds) | tools (44+) | plugins  |
+-----+
|  api (6+ providers)   | runtime (core engine)  |
|  enterprise (circuit breaker, health, retry)   |
+-----+
| SECURITY HARDENING (Fases 1-9, ~39K lineas)    |
| heuristic_engine | circuit_breaker | health    |
| sanitizer | path_traversal | fingerprint    |
| rate_limiter | concurrency | size_budget    |
| audit_integration | forensic | siem_export    |
| self_healing | recovery_recipes | worker_boot |
+-----+
| vulnscan (9 langs) | osint (8 mods) | security |
+-----+
| localmodels | sandbox | cache | offline | ... |
+-----+
```

2.2. Grafo de Dependencias

Las dependencias internas siguen una jerarquia en estrella donde runtime (~39,173 lineas, 57 archivos) es el hub central. La cadena de dependencia fluye:

security/telemetry -> plugins -> runtime -> api/commands -> tools -> rusty-clang-cli

- plugins, security, telemetry: crates hoja sin dependencias internas
- runtime: hub central, depende de plugins, security, telemetry
- api: depende de runtime y telemetry
- osint, enterprise, cache, offline, optimization, sandbox: autonomos (solo dependencias externas)
- vulnscan: depende de runtime, security, localmodels, api

2.3. Estructura por Crate

Crate	Lineas	Archivos	Proposito
runtime	39,173	57	Core engine: sesiones, permisos, MCP, heuristic_engine, circuit_breaker, self_healing, audit
rusty-clang-cli	15,584	4	Binario principal: REPL, CLI, parser de argumentos
tools	11,003	3	44+ herramientas del agente (read, write, edit, bash, search, plan)
api	7,931	10	Clientes multi-provider (Anthropic, OpenAI, DeepSeek, xAI, Ollama)
vulnscan	7,686	41	Escaner 9 lenguajes, LLM analyst, BFS, exploits, reportes
commands	5,976	1	140+ comandos slash, parseo, ayuda
osint	4,539	11	OSINT: DNS, email, social, personas, infra, reportes
plugins	4,294	3	Sistema de plugins con ciclo de vida completo

sandbox	2,612	9	Seccomp BPF, Landlock, namespace, tmpfs, rlimit, NSJail, macOS, Windows
security	1,879	7	AES-256-GCM, XChaCha20Poly1305, Argon2id, Ed25519 audit
localmodels	1,852	7	ML: 66 features, classifier 3-class, softmax, ensemble, online SGD
enterprise	1,820	10	Circuit breaker, retry, health checks, metrics, tracing
cache	1,228	1	Cache multi-nivel (mem+SQLite), LRU/LFU/FIFO/TTL, zlib
optimization	1,192	5	PSO, GA, ACO, Simulated Annealing
mock-anthropic	1,157	2	Mock determinista Anthropic para tests
telemetry	526	1	Tipos de datos de telemetria
compat-harness	367	1	Tests de paridad de comportamiento
offline	232	3	Cola SQLite con auto-sync

2.4. Flujo de Datos Transversal

Cada solicitud (tool call) atraviesa el siguiente pipeline de seguridad en orden:

- 1. Sanitizer Pipeline (7 etapas): normalizacion -> canonicalizacion -> resolucion symlinks -> scope check -> deteccion encoding -> size check -> allowlist
- 2. Path Traversal Matrix (8 tecnicas): directory traversal, double encoding, unicode, proc/self/fd, null byte, device files, FIFO, Windows ADS
- 3. Size Budgeter: limites por tool (ventana 5min) + limites de sesion (5K calls, 200MB)
- 4. Concurrency Manager: semaforos por categoria (bash=5, read=20, write=3, search=2, MCP=10)
- 5. Heuristic Engine (55+ reglas): scoring heuristico con perfil de comportamiento
- 6. Classifier ML (66 features): regresion logistica 3-class + ensemble (heuristica 0.4 + ML 0.4 + secuencia 0.2)
- 7. Permission Enforcer: evalua politicas, promptea al usuario o bloquea
- 8. Circuit Breaker: verifica estado del provider/servicio (Closed/Open/HalfOpen)
- 9. Rate Limiter: token bucket adaptativo por provider + global
- 10. Sandbox: seccomp BPF (~205 syscalls read-only, ~27 write) + Landlock (paths RO/RW) + namespace + rlimit
- 11. Auditoria: hash chain SHA-256 + bloques firmados Ed25519 cada 100 entradas
- 12. Self-Healing: checkpoint cada 5 tool calls, WAL, health monitor, auto-restart

3. Analisis de Rendimiento

3.1. Limites de Recursos por Tool

El sistema impone limites estrictos de recursos por herramienta y por sesion para prevenir abuso y agotamiento de recursos.

Tool	Bytes Max	Calls Max	Ventana
Read	10 MB	200	300s
Write	5 MB	100	300s
Edit	5 MB	100	300s
Glob	1 MB / 1K entries	50	300s
Grep	1 MB	100	300s
Bash	10 MB	100	300s
Sesion total	200 MB	5,000	vida

3.2. Limites de Concurrencia

Categoria	Limite	Descripcion
Bash	5	Ejecuciones bash simultaneas
Read	20	Lecturas de archivo simultaneas
Write	3	Escrituras de archivo simultaneas
Search	2	Busquedas simultaneas
MCP	10	Llamadas MCP simultaneas

3.3. Sandbox Resource Limits (rlimit)

Recurso	Valor	Proposito
CPU time	60s soft / 90s hard	Evita loops infinitos
Address space (AS)	1 GB	Previene OOM
File size (FSIZE)	100 MB	Limita escrituras
Open files (NOFILE)	256	Previene fd exhaustion
Processes (NPROC)	16	Previene fork bombs

3.4. Latencia y Timeouts

- Health probe interval: 5s por provider
- Timeout default health probe: 10,000ms (30,000ms para Ollama)
- Degraded: p95 > 5,000ms o p99 > 10,000ms o error_rate > 10%
- Unhealthy: consecutivos >= 5 o p99 > 10,000ms o error_rate > 50%
- MCP initialize timeout: 10,000ms
- MCP list tools timeout: 30,000ms
- MCP tool call timeout: 60,000ms

- Provider retry backoff: 1s -> 2s -> 4s -> ... -> 128s (max 8 retries)
- ML inference latency target: < 1ms
- Heuristic scoring latency target: < 2ms
- Latencia total anadida por capas de seguridad: < 100ms

3.5. Rate Limiting Adaptativo

El rate limiter usa un token bucket adaptativo que se ajusta cada 30 segundos:

- Base capacity: 60 tokens por provider; global: 200 tokens
- Refill rate: 1 token/s por provider; 3 token/s global
- Rango dinamico: min 10, max 120 por provider; min 50, max 400 global
- Confidence bonus: +10% si error_rate < 5%, max +50%
- Error malus: -20% si error_rate > 20%, max -80%
- Decay: -5% bonus, -2.5% malus cuando error_rate entre 5-20%

3.6. Benchmarks

El unico benchmark formal usa criterion.rs en el crate api (4 grupos, 24 variantes):

- translate_message: texto, asistent+tools, tool_result, kimi-k2.5, texto grande (10KB)
- build_chat_completion_request: 10, 50, 100 mensajes; o1-mini, gpt-5
- flatten_tool_result_content: 1, 10, 50 bloques; JSON, mixto
- is_reasoning_model: 7 variantes de modelo (gpt-4o, o1, grok, qwen)

4. Analisis de Seguridad

4.1. Mapa de Fases de Hardening

9 de 10 fases completadas del roadmap de seguridad defensiva en profundidad:

Fase	Nombre	Estado	Implementacion
1	Fortaleza Criptografica	[OK]	Argon2id (64MB, t=3, p=4), XChaCha20Poly1305, AES-256-GCM, zeroize, mlock
2	Input Fortress + Fuzzing	[OK]	Sanitizer 7-etapas, Path Traversal 8 tecnicas, fuzzing continuo
3	Heuristic Anomaly Engine	[OK]	55+ reglas, 3 ventanas temporales, feedback loop online
4	Circuit Breakers + Rate Limiting	[OK]	Arbol jerarquico 4 niveles, token bucket adaptativo
5	Audit Fort Knox + SIEM	[OK]	Hash chain SHA-256, Ed25519 signing, ECS/Splunk/OTel
6	Sandbox Real	[OK]	Seccomp BPF, Landlock ABI 1-2, 8 namespaces, rlimit
7	ML Local Detection	[OK]	66 features, logistic regression 3-class, SGD online learning
8	Supply Chain Fortress	[OK]	cargo-deny, cargo-audit, SBOM, 5 fuzz targets, vendoring
9	Self-Healing Immune System	[OK]	Checkpoint WAL, health monitor, auto-restart, graceful shutdown
10	Adaptive Security Engine	[..]	Pendiente: policy evolution, threat intel, honeytokens

4.2. Heuristic Engine (55+ Reglas)

El motor heuristico compila 55+ reglas de seguridad en 10 categorias con pesos y severidades configurables. El scoring combina reglas ponderadas con desviacion de perfil de comportamiento.

- Tool-based (9): write-to-system-path, rm-root, chmod-dangerous, device-write, proc-fs-write, etc.
- Network/Exfiltration (1): network-exfiltration
- Privilege escalation (2): sudo-command, ssh-command
- Fork bomb (1): fork-bomb
- Intent-based (5): network-in-readonly, destructive-in-workspace, etc.
- Context-aware (7): rm-in-build-dir, read-sensitive-config, read-ssh-keys, etc.
- Behavioral (6): recon-read-burst, scan-chain, write-burst, high-error-rate, etc.
- Combined/Chained (3): read-then-network, crypto-miner-patterns, etc.
- Known attacks (11): reverse-shell, docker-escape, mass-deletion, curl-pipe-bash, etc.
- System manipulation (5): firewall-changes, user-management, cron-manipulation, etc.

4.3. Risk Score Thresholds

Nivel	Rango Score	Accion
Safe	< 0.30	Sin accion
Low	0.30 - 0.59	Monitoreo
Medium (warn)	0.60 - 0.79	Advertir al usuario
High (prompt)	0.80 - 0.94	Solicitar confirmacion
Critical (block)	>= 0.95	Bloquear operacion

4.4. Clasificador ML (66 Features)

El clasificador de regresion logistica multinomial usa 66 features extraidas de cada comando bash en 12 categorias. Usa softmax 3-class (benigno/sospechoso/malicioso) con pesos hand-tuned.

Categoria	Cantidad	Features
Length & entropy	5	char_len, word_count, avg_word_len, entropy, max_token_len
Shell syntax	8	pipe, semicolon, backtick, var_sub, redirect, heredoc, subshell, logical_op
Dangerous patterns	10	rm_rf, dd, mkfs, chmod -R, chown, wget/curl, eval, bash network
Encoding anomalies	5	base64, hex, rot13, url_encoding, unicode_escape
Path analysis	8	/etc/, /proc/, /dev/, .ssh/, .git/, dot dir, /tmp/, ~/
Network indicators	6	IP, URL, port, domain, localhost, network connection
Process manip	6	kill, nohup, disown, bg, fork bomb, ptrace
Privilege escalation	5	sudo, su, setuid, capabilities, pkexec
Persistence	4	cron, systemd, rc.local, ssh_authorized_keys
Compression	3	tar, zip, gzip
Compilation	3	gcc, make, cc
Ratios	3	special_char_ratio, digit_ratio, uppercase_ratio

4.5. Pesos del Modelo ML (Default)

Los pesos mas altos detectan actividad maliciosa: bash_network (3.5), fork_bomb (3.5), rm_rf (3.0), wget/curl pipe (3.0), eval (2.5), base64 (2.5), ssh_path (2.5), pkexec (2.5). Biases: benign_bias=0.0, suspicious_bias=-1.0, malicious_bias=-1.5.

4.6. Sandbox Isolation

- Seccomp BPF: ~205 syscalls permitidas en modo read-only, ~27 adicionales para write, 5 prohibidas (ptrace, perf_event_open, bpf, process_vm_writev, kcmp, execveat)
- Landlock: ABI 1 y 2, reglas path-based con access_fs_read/access_fs_rw, restriccion irreversible via landlock_restrict_self
- Namespace: 8 tipos aislables (User, Mount, PID, Net, IPC, UTS, Cgroup, Time). Default: User+Net+IPC+UTS
- Resource limits: CPU 60s, AS 1GB, FSIZE 100MB, NOFILE 256, NPROC 16
- NSJail: perfil completo con chroot, tmpfs, cpus, memoria, time limits
- Cross-platform: macOS (Seatbelt sandbox_init), Windows (JobObject + AppContainer)

4.7. Supply Chain Security

- cargo-deny: allowlist de 8 licencias, denylist copyleft, vulnerability db, source bans
- cargo-audit: contra RustSec Advisory Database en CI
- SBOM CycloneDX: generado via cargo-cyclonedx --all en cada tag push
- Fuzzing semanal: 5 targets x 5 min (sanitizer, path_traversal, bash_validation, features, json_config)
- Unsafe audit: escaneo de unsafe en rust/crates/ con reporte en CI
- Vendoring: cargo vendor para builds air-gapped
- Politica documentada: SUPPLY-CHAIN.md con incident response plan

4.8. Self-Healing Immune System

- SessionCheckpoint: snapshot + WAL cada 5 tool calls o 60s. MAX_WAL=100. Prune keep-last-3
- HealthMonitor: background thread cada 5s, heartbeats por componente, memoria (free<5% critico), disco (free<2% critico)
- AutoRestarter: backoff exponencial 1s->120s, multiplicador 2.0, jitter 10%, max 5 intentos por componente
- CorruptionRepair: SHA-256 checksums, fallback chain WAL -> snapshot -> fresh
- GracefulShutdown: signal handlers SIGTERM/SIGINT, flush audit, checkpoint, close MCP, zeroize

5. Analisis de Sensibilidad

5.1. Thresholds Criticos del Sistema

El sistema utiliza multiples umbrales configurables que determinan su comportamiento ante diferentes condiciones de carga, fallo y ataque.

Umbral	Valor	Efecto
Health Degraded	p95 > 5s o p99 > 10s	Latencia alta del provider
Health Unhealthy	p99 > 10s o error_rate > 50%	Provider caido
Circuit Open	3-5 fallos consecutivos	Fallo del servicio
Circuit Recovery	2 exitos consecutivos	Vuelta a la normalidad
Rate Limiter Cap	10-120 tokens/provider	Proteccion contra abuso
Risk Safe	< 0.30	Operacion normal
Risk Block	>= 0.95	Ataque detectado
Memory Critical	free < 5%	Riesgo de OOM
Disk Critical	free < 2%	Riesgo de disco lleno
WAL Overflow	100 entradas	Forza checkpoint completo
Backoff Max	120s	Tiempo max entre reintentos
Session Budget	200 MB / 5K calls	Limite de sesion

5.2. Analisis de Falsos Positivos / Negativos

El sistema tiene mecanismos de ajuste automatico de sensibilidad:

- Feedback Loop: ajusta pesos de reglas +/-0.2 basado en approve/reject del usuario. Minimo 3 muestras, estabilidad tras 10
- Rate Limiter Adaptive: ajusta capacidad cada 30s. Si error_rate < 5%, sube capacidad. Si > 20%, baja
- Circuit Breaker: ajusta umbrales por nodo. Los thresholds son jerarquicos (3-5 fallos segun nivel)
- Behavioral Profile: ventanas de 5min/30min/2h para detectar desviaciones de linea base
- Ensemble Scoring: combina heuristica (0.4) + ML (0.4) + secuencia (0.2) para reducir falsos positivos

5.3. Cobertura de Deteccion

El sistema cubre 18 clases de vulnerabilidad OWASP/CWE con deteccion estatica + LLM, mas 55+ reglas heurísticas comportamentales y un clasificador ML de 66 features. KPIs objetivo:

- Deteccion de ataques heurísticos: > 95% con < 5% FP (Fase 3)
- Recall de deteccion ML: > 90% (Fase 7)
- 0 path traversal bypass en 24h de fuzzing (Fase 2)
- 0 sandbox escapes en 20 tecnicas conocidas (Fase 6)
- Recovery de crash: < 1s, 0 perdida de datos (Fase 9)

6. KPIs y Metricas

6.1. Resumen de KPIs por Fase

Fase	KPI Principal	KPI Secundario
1	0 secretos en texto plano	< 5ms encriptacion
2	0% path traversal bypass	Fuzzing 24h
3	> 95% deteccion, < 5% FP	< 2ms decision
4	0 fallos en cascada	89 nuevos tests
5	100% tool calls auditadas	39 nuevos tests
6	0 sandbox escapes	51 tests
7	> 90% recall ML	< 1ms inferencia, 66 tests
8	0 advisories criticos	5 fuzz targets, SBOM
9	Recovery < 1s	0 data loss, 21 tests
10	FP < 3%, FN < 5%	< 100ms respuesta incidentes

6.2. Cobertura de Tests

Total: 1,594 tests registrados en el workspace.

Crate	Tests	Area
runtime	702	Core engine + seguridad
rusty-claude-cli	213	CLI, integracion
api	160	Cientes API, serializacion
tools	95	Herramientas del agente
osint	79	OSINT framework
localmodels	66	ML detection
sandbox	51	Sandbox isolation
security	46	Criptografia + auditoria
cache	45	Cache multi-nivel
commands	41	Comandos slash
plugins	40	Sistema de plugins
enterprise	27	Enterprise features
optimization	12	Algoritmos de optimizacion
vulnscan	10	Escaner vulnerabilidades
telemetry	3	Telemetria
compat-harness	3	Compatibilidad
offline	1	Modo offline

6.3. KPIs de Rendimiento

- Latencia de encriptacion (Argon2id + XChaCha20Poly1305): < 5ms por operacion
- Latencia de decision heuristica: < 2ms (55+ reglas compiladas)
- Latencia de inferencia ML: < 1ms (66 features, softmax 3-class)
- Latencia total anadida por capas de seguridad: < 100ms
- Checkpoint de sesion: cada 5 tool calls o 60s
- Health monitor poll: cada 5s
- Rate limiter adjustment: cada 30s
- Recovery de crash: < 1s
- Provider retry: hasta 8 reintentos con backoff 1s-128s
- Cache: multi-nivel (memoria + SQLite), algoritmos LRU/LFU/FIFO/TTL

6.4. Volumen deCodigo

- Total: 111,053 lineas de Rust en 176 archivos fuente
- Tests: 1,594 tests registrados
- Binary (debug): ~225 MB
- Build artifacts (debug target): ~38 GB
- Workspace: 18 crates (2 binarios, 16 librerias)
- Dependencias externas: ~200 crates
- Benchmarks: 4 grupos criterion (24 variantes)
- Fuzz targets: 5 (semanal, 5 min cada uno)
- Scripts: 3 (sbom, vendor, chaos-test)

6.5. Thresholds de Seguridad por Modo

PermissionMode	Risk Threshold	Comportamiento
DangerFullAccess	0	Sin restricciones
AcceptAll	N/A	Permisos automaticos con auditoria
AcceptEditsAndReads	N/A	Solo comandos de escritura requieren permiso
Strict	0.60	Warn en Medium, prompt en High

7. Conclusion

Kraken representa un sistema de agente IA con el nivel mas alto de hardening de seguridad implementado en Rust. Con 9 de 10 fases completadas, el sistema ofrece:

- Defensa en profundidad: 12 capas de seguridad desde el sanitizer de input hasta el sandbox de ejecucion, pasando por deteccion ML, circuit breakers y auto-recuperacion
- Zero trust: todos los comandos se evaluan, clasifican, puntuan y autorizan antes de ejecutarse
- Auto-recuperacion: el sistema puede detectar fallos, checkpointear estado, reiniciar componentes y recuperarse sin perdida de datos
- Supply chain segura: dependencias verificadas, SBOM generado, fuzzing continuo, vendoring para air-gapped
- Auditabilidad total: cada accion se registra en una cadena de hash SHA-256 con firmas Ed25519, exportable a SIEM en formatos ECS, Splunk HEC y OpenTelemetry
- Multi-proveedor gratuito: funcionamiento sin suscripciones de pago usando DeepSeek, Ollama, Big Pickle y otros
- Portabilidad: un solo binario estatico de Rust que funciona en Linux, macOS y Windows

Proxima fase (Fase 10): Adaptive Security Engine con auto-evolucion de politicas, threat intelligence feeds, honeytokens y ajuste automatico de umbrales.